IPP - Practical File 1

Name: Swanand Wirkar
Course: BDA
Enrollment Number: 20251051

Question: Write a Python program to read an image named "flower.jpg" using OpenCV and display it in a window titled "Original Image".

Code:

```
import cv2
image = cv2.imread("flower.jpg")
cv2.imshow("Original Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. import cv2 imports the OpenCV library, which is used for image processing.
2. cv2.imread("flower.jpg") reads the image file and stores its pixel data in image.
3. cv2.imshow("Original Image", image) displays the image in a window with the title Original Image.
4. cv2.waitKey(0) waits until the user presses a key.
5. cv2.destroyAllWindows() closes all OpenCV display windows.

Question: Write a program to print the dimensions (height, width, channels) and data type of an image named "scene.jpg".

Code:

```
import cv2
image = cv2.imread("scene.jpg")
height, width, channels = image.shape
print("Height:", height)
print("Width:", width)
print("Channels:", channels)
print("Data type:", image.dtype)
```

Explanation:

1. import cv2 imports OpenCV for reading and processing images.
2. cv2.imread("scene.jpg") loads scene.jpg into the variable image.
3. image.shape returns the image dimensions as height, width, and channels for a colour image.
4. The next three print statements display the height, width, and number of colour channels.
5. image.dtype returns the data type used to store each pixel value, such as uint8.

Note: This code assumes the image loads successfully and is a colour image.

Question: Write a Python program to load a colour image and convert it into a grayscale image, then save the result as "output_gray.jpg".

Code:

```
import cv2
image = cv2.imread("flower.jpg")
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imwrite("output_gray.jpg", gray_image)
```

Explanation:

1. import cv2 imports the OpenCV library.
2. cv2.imread("flower.jpg") loads the colour image. OpenCV reads colour images in BGR channel order by default.
3. cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) converts the BGR colour image into a single-channel grayscale image.
4. cv2.imwrite("output_gray.jpg", gray_image) saves the grayscale image with the filename output_gray.jpg.

The input file must be in the program's working folder, or its full path must be provided.

Question: Write a short program to open "sunset.png" using Pillow and display it.

Code:

```
from PIL import Image
image = Image.open("sunset.png")
image.show()
```

Explanation:

1. from PIL import Image imports the Image module from the Pillow library.
2. Image.open("sunset.png") opens the image file and stores it in image.
3. image.show() opens the image in the system's default image viewer.

Pillow must be installed, and sunset.png must be available at the given path.

Question: Write a Python program using OpenCV that resizes an image to 50% of its original width and height.

Code:

```
import cv2
image = cv2.imread("flower.jpg")
height, width = image.shape[:2]
resized_image = cv2.resize(image, (width // 2, height // 2))
cv2.imshow("Resized Image", resized_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. import cv2 imports OpenCV.
2. cv2.imread("flower.jpg") loads the original image.
3. image.shape[:2] gets only the height and width, leaving out the channel count.
4. width // 2 and height // 2 calculate half the original dimensions using integer division.
5. cv2.resize(image, (width // 2, height // 2)) creates a smaller image at 50% of the original width and height.
6. cv2.imshow("Resized Image", resized_image) displays the resized image.
7. cv2.waitKey(0) waits for a key press, and cv2.destroyAllWindows() closes the display window.

Question: Write a Python code snippet to crop the central 200×200 pixel area of an image and display it using Matplotlib.

Code:

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("flower.jpg")
height, width = image.shape[:2]
x = (width - 200) // 2
y = (height - 200) // 2
cropped_image = image[y:y + 200, x:x + 200]
cropped_image = cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB)
plt.imshow(cropped_image)
plt.axis("off")
plt.show()
```

Explanation:

1. import cv2 imports OpenCV to read and process the image.
2. import matplotlib.pyplot as plt imports Matplotlib's plotting functions and gives them the short name plt.
3. cv2.imread("flower.jpg") loads the image.
4. image.shape[:2] obtains the image height and width.
5. The x calculation finds the left starting coordinate of the central crop.
6. The y calculation finds the top starting coordinate of the central crop.
7. image[y:y + 200, x:x + 200] selects a 200-pixel-high and 200-pixel-wide region.
8. cv2.cvtColor(..., cv2.COLOR_BGR2RGB) changes the channel order because OpenCV uses BGR while Matplotlib expects RGB for colour images.
9. plt.imshow(cropped_image) displays the cropped image.
10. plt.axis("off") hides the coordinate axes.
11. plt.show() renders the plot.

The source image should be at least 200×200 pixels.

Question: Write a short program that rotates an image by 90 degrees clockwise using OpenCV.

Code:

```
import cv2
image = cv2.imread("flower.jpg")
rotated_image = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)
cv2.imshow("Rotated Image", rotated_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. import cv2 imports OpenCV.
2. cv2.imread("flower.jpg") reads the image file.
3. cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE) rotates the image 90 degrees clockwise and stores the result in rotated_image.
4. cv2.imshow("Rotated Image", rotated_image) displays the rotated image.
5. cv2.waitKey(0) keeps the window open until a key is pressed.
6. cv2.destroyAllWindows() closes the OpenCV windows.

Question: Write a Python statement to flip the image "car.jpg" left-to-right using OpenCV and display the result.

Code:

```
import cv2
image = cv2.imread("car.jpg")
flipped_image = cv2.flip(image, 1)
cv2.imshow("Flipped Image", flipped_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. import cv2 imports OpenCV.
2. cv2.imread("car.jpg") loads car.jpg.
3. cv2.flip(image, 1) flips the image horizontally; the value 1 means left-to-right.
4. cv2.imshow("Flipped Image", flipped_image) displays the flipped result.
5. cv2.waitKey(0) waits for a key press.
6. cv2.destroyAllWindows() closes all OpenCV display windows.

Question: Write a Python program that converts an image from RGB to HSV colour space, and display each channel (Hue, Saturation, Value) separately using Matplotlib.

Code:

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("flower.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
hsv_image = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)
hue, saturation, value = cv2.split(hsv_image)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(hue, cmap="gray")
axes[0].set_title("Hue")
axes[1].imshow(saturation, cmap="gray")
axes[1].set_title("Saturation")
axes[2].imshow(value, cmap="gray")
axes[2].set_title("Value")
for axis in axes:
    axis.axis("off")
plt.tight_layout()
plt.show()
```

Explanation:

1. import cv2 imports OpenCV for image processing.
2. import matplotlib.pyplot as plt imports plotting tools from Matplotlib.
3. cv2.imread("flower.jpg") reads the image in BGR format.
4. cv2.cvtColor(image, cv2.COLOR_BGR2RGB) converts the image from BGR to RGB.
5. cv2.cvtColor(image, cv2.COLOR_RGB2HSV) converts the RGB image into HSV colour space.
6. cv2.split(hsv_image) separates the HSV image into Hue, Saturation, and Value channels.
7. plt.subplots(1, 3, figsize=(12, 4)) creates one row with three plotting areas.
8. Each imshow statement displays one channel in grayscale, and each set_title statement labels that channel.
9. The for loop hides the axes for all three plots.
10. plt.tight_layout() adjusts spacing between the plots.
11. plt.show() displays all three channels together.

Question: Write a short Python code snippet to compute the Mean Squared Error (MSE) between two grayscale images, "img1.jpg" and "img2.jpg".

Code:

```
import cv2
import numpy as np
image1 = cv2.imread("img1.jpg", cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread("img2.jpg", cv2.IMREAD_GRAYSCALE)
mse = np.mean((image1.astype(np.float64) - image2.astype(np.float64)) ** 2)
print("MSE:", mse)
```

Explanation:

1. import cv2 imports OpenCV to read the image files.
2. import numpy as np imports NumPy for numerical calculations.
3. The first cv2.imread statement loads img1.jpg as a grayscale image.
4. The second cv2.imread statement loads img2.jpg as a grayscale image.
5. astype(np.float64) converts pixel values to a floating-point type so subtraction and squaring do not overflow the original 8-bit pixel type.
6. The subtraction calculates the difference between corresponding pixels in the two images.
7. ** 2 squares each pixel difference.
8. np.mean(...) calculates the average of all squared differences, which is the Mean Squared Error.
9. print("MSE:", mse) displays the calculated MSE.

Both images must load successfully and have the same dimensions. An MSE of 0 means the images have identical pixel values.